OPERATING SYSTEMS – Task 2

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem (Semaphores and Monitors)

Name: M G Krupa | USN: NNM24IS116 | Course: Operating Systems |

GitHub repository: https://github.com/mgkrupa06/OS_Task2

1. Problem Statement and Synchronization Objective

When multiple threads share data we don't control the order in which they run. If they aren't coordinated the outcome can change depending on timing. This is called a race condition. In this assignment we solve two synchronization problems. Readers-Writers and Dining Philosophers. Each using two methods: semaphores and monitors. That makes four programs total.

Readers-Writers. Many readers can read a shared resource at the time.. A writer must have the resource all to themselves. The goal is to make sure a writer never runs at the time as a reader or another writer. Readers can still share the resource. Also no one should wait forever. Everyone must eventually get what they need.

Dining Philosophers (4 philosophers). Four philosophers sit around a table. There are four forks, one between each pair of neighbors. To eat a philosopher needs both forks next, to them. The goal is to make sure each fork is used by one philosopher at a time. We must avoid deadlock. Where everyone is waiting. No one can eat. We also need to talk about starvation. When someone waits long and never gets to eat.

In addition every program must create an HTML simulation. This simulation must come from the data of the program's real run not from a hand-drawn or made-up animation.

2. Implementation Overview

All programs are written in Python 3 using only the standard library (threading, json, webbrowser). Each philosopher, reader and writer is a real thread. Random sleep times are used only to make the interleaving visible; they never replace the synchronization primitives.

- **readers_writers/semaphore/rw_semaphore.py** - Readers-Writers with semaphores
- **readers_writers/monitor/rw_monitor.py** - Readers-Writers with a monitor
- **dining_philosophers/semaphore/dp_semaphore.py** - 4 philosophers with semaphores
- **dining_philosophers/monitor/dp_monitor.py** - 4 philosophers with a monitor
- **common/sync_trace.py** - shared module that records events, checks them and generates the HTML

There is no compile step. Run for example python readers_writers/semaphore/rw_semaphore.py --readers 4 --writers 2 --rounds 3. The options --delay, --seed and --out change the speed, make timing repeatable and choose the HTML path. After the run, the HTML file is written to generated_html/ and opened in the browser.

3. Readers-Writers - Semaphore Solution

Shared resource: a SharedDatabase object holding one integer. Its write() method reads the value, sleeps briefly, then stores value + 1. This is deliberately not atomic: if two writers, or a writer and a reader, were ever inside together, updates would be lost or a wrong value would be read.

Critical sections: the body of a reader (reading the value) and the body of a writer (db.write()).

Semaphores and initial values:

- **resource_lock (1):** held by one writer, or by the whole group of active readers together.
- **count_mutex (1):** protects the shared variable active_readers, because the counter is itself shared data.
- **turnstile (1):** a gate that every reader and writer passes through when requesting access. Since arrivals queue at this gate, a waiting writer blocks readers that arrive later, and readers cannot overtake it forever.

Reader: wait(turnstile), wait(count_mutex), increment active_readers; if it became 1, wait(resource_lock) so writers are locked out; signal(count_mutex), signal(turnstile). Read. On exit: wait(count_mutex), decrement; if it became 0, signal(resource_lock); signal(count_mutex).

Writer: wait(turnstile), wait(resource_lock), signal(turnstile). Write. Then signal(resource_lock).

The first reader acquires the lock on behalf of all readers and the last reader releases it, which is how readers share while writers are excluded.

4. Readers-Writers - Monitor Solution

Python has no monitor keyword, so a monitor is built as a class (ReadersWritersMonitor) whose shared state is only touched while holding one lock. Waiting is done with two condition variables created on that same lock, which is the allowed equivalent abstraction.

Monitor state: active_readers, writing, waiting_writers. Condition variables: ok_to_read and ok_to_write.

- **start_read:** waits until no writer is writing and no writer is waiting, then increments active_readers.
- **end_read:** decrements active_readers; if zero, notifies one waiting writer.
- **start_write:** increments waiting_writers, waits until nobody is reading or writing, then sets writing = True.
- **end_write:** clears writing; if writers are waiting it wakes one, otherwise it wakes all waiting readers.

Because Python uses Mesa-style conditions (a woken thread must check again), the waits use wait_for(condition), which re-checks the predicate after waking. The actual reading and writing happen outside the entry and exit procedures, so readers can run in parallel. The policy is writer preference.

5. Dining Philosophers (4) - Semaphore Solution

Philosopher i uses fork i (left) and fork (i+1) mod 4 (right). Each fork is a binary semaphore with initial value 1, so a fork can belong to only one philosopher at a time. That gives mutual exclusion on the forks.

If every philosopher picks up the left fork and then waits for the right one, all four hold one fork and wait forever, which is a circular wait, so deadlock. To prevent this, a counting semaphore room is initialised to 3 (N - 1). A philosopher must wait(room) before touching any fork, and signal(room) after eating. At most three philosophers compete at once, so at least one of them can always get both forks, and the circular wait cannot form.

Cycle of each philosopher: think, become hungry, wait(room), wait(left), wait(right), eat, signal(left), signal(right), signal(room).

6. Dining Philosophers (4) - Monitor Solution

The DiningTable monitor keeps a state for each philosopher (THINKING, HUNGRY, EATING) and one condition variable per philosopher, all on one lock.

- **pickup(i):** set state HUNGRY, call _test(i), and wait on can_eat[i] until the state becomes EATING.

- **_test(i):** if i is hungry and neither neighbour is eating, set EATING (this takes both forks at once) and notify i.
- **putdown(i):** set THINKING, release both forks, then call _test for the left and right neighbours, because one of them may now be able to eat.

The check and the change of state happen atomically inside the monitor, so a philosopher takes both forks or none. Hold-and-wait, one of the four conditions for deadlock, never happens, so deadlock is impossible.

7. How the Program Records Events and Generates the HTML

1. Every thread calls Tracer.log(actor, state, detail, ...) each time its state changes, for example waiting, reading, writing, hungry, acquiring, eating, releasing. For philosophers the call also carries the fork number and its new owner (or none when released).
2. log() takes a lock, so every event gets a sequence number and a time stamp in one single order. Events are logged while the thread holds the relevant semaphore or monitor lock (and before it releases it), so the recorded order cannot show an impossible overlap.
3. After all threads finish, the program replays the trace and asserts the safety rules: no reader/writer or writer/writer overlap, no fork with two owners, and every eating philosopher holds both of its forks.
4. write_html() embeds the event list as JSON inside a self-contained HTML page. JavaScript in the page applies the events one by one: it colours readers, writers and philosophers by state, shows whether the shared resource is free, shared-read or exclusive-write, draws the four forks and who holds them, and prints the event log. Start, Pause, Step, Reset, a speed slider and a timeline slider allow replay.

The HTML contains no hard-coded story; if the program is run again with different parameters, a different file is generated.

8. Execution Traces (from real runs)

These are excerpts from runs with --seed 3 --delay 0.3. A run without the seed will give a different but equally valid order.

8.1 Readers-Writers, semaphores (3 readers, 1 writer)

```
[0.148s] R3 waiting  requests read access
[0.148s] R3 reading  entered CS, readers inside = 1
[0.276s] R2 reading  entered CS, readers inside = 2
[0.346s] R4 reading  entered CS, readers inside = 3
[0.416s] R3 idle     left CS, readers inside = 2
[0.536s] W2 waiting  requests write access
[0.540s] R3 waiting  requests read access
[0.575s] R1 waiting  requests read access
[0.588s] W1 waiting  requests write access
[0.599s] R2 idle     left CS, readers inside = 1
[0.743s] R4 idle     left CS, readers inside = 0
[0.744s] W2 writing  entered CS (exclusive)
```

Up to three readers are inside the critical section together. W2 asks to write at 0.536s, and the readers that arrive afterwards (R3, R1) and the writer W1 have to wait behind it, so the new readers do not overtake the waiting writer. W2 enters only at 0.744s, one millisecond after the last reader (R4) leaves and readers inside drops to 0.

8.2 Readers-Writers, monitor (same settings)

```
[0.112s] R1 reading  entered CS, readers inside = 1
[0.146s] R2 reading  entered CS, readers inside = 2
```

```
[0.423s] R3 reading  entered CS, readers inside = 3
[0.484s] R4 reading  entered CS, readers inside = 4
[0.509s] R2 idle     left CS, readers inside = 3
[0.598s] R1 idle     left CS, readers inside = 2
[0.782s] W2 waiting  requests write access
[0.819s] R3 idle     left CS, readers inside = 1
[0.839s] R4 idle     left CS, readers inside = 0
[0.839s] W2 writing  entered CS (exclusive)
```

All four readers are inside at the same time, which shows that readers share the resource. W2 asks to write at 0.782s while R3 and R4 are still reading, so it waits. It enters at 0.839s, the same instant the last reader leaves and the count reaches 0.

8.3 Dining Philosophers, semaphores

```
[4.299s] P2 releasing  put down fork 2
[4.300s] P2 releasing  put down fork 3
[4.301s] P2 thinking   finished all meals
[4.301s] P1 acquiring  picked up fork 2
[4.302s] P1 eating     has both forks
[4.632s] P1 releasing  put down fork 1
[4.633s] P1 releasing  put down fork 2
[4.634s] P1 thinking   finished all meals
[4.635s] P0 acquiring  picked up fork 1
[4.635s] P0 eating     has both forks
[4.943s] P0 releasing  put down fork 0
[4.944s] P0 releasing  put down fork 1
[4.944s] P0 thinking   finished all meals
validation OK: forks never shared and every meal had both forks
```

P1 can eat only after P2 puts down fork 2, and P0 can eat only after P1 puts down fork 1. Each waiting philosopher starts eating about a millisecond after its neighbour releases the fork, so the forks are passed on in turn. All philosophers finished all their meals, so there was no deadlock, and the program's own check printed validation OK.

8.4 Dining Philosophers, monitor

```
[0.150s] P0 hungry     wants to eat
[0.156s] P0 acquiring  took fork 0
[0.157s] P0 acquiring  took fork 1
[0.157s] P0 eating     has both forks
[0.158s] P2 hungry     wants to eat
[0.158s] P2 acquiring  took fork 2
[0.158s] P2 acquiring  took fork 3
[0.159s] P2 eating     has both forks
[0.186s] P3 hungry     wants to eat
[0.533s] P1 hungry     wants to eat
[0.609s] P2 releasing  put down fork 2
[0.634s] P2 releasing  put down fork 3
[0.634s] P2 thinking   back to thinking
[0.634s] P0 releasing  put down fork 0
[0.634s] P0 releasing  put down fork 1
```

P0 and P2 eat at the same time because they are not neighbours, and each takes both of its forks together. P3 and P1 become hungry but do not take any forks while a neighbour is eating (P3 sits between P2 and P0, and P1 between P0 and P2). They can only proceed after P2 and P0 put their forks down.

9. Race Conditions, Mutual Exclusion, Deadlock and Starvation

Race condition. In Readers-Writers, SharedDatabase.write() is a read-modify-write with a sleep in the middle. Without the exclusive writer lock, two writers could both read the same old value and one increment would be lost. The final-value check (final value equals writers x rounds) and the trace validation confirm this did not happen. The counter active_readers would also race if it were not guarded by count_mutex (or the monitor lock).

Mutual exclusion. Readers-Writers: resource_lock (or the monitor condition) keeps a writer apart from everyone else. Philosophers: each fork semaphore (or the monitor state check) lets only one philosopher hold a fork.

Deadlock. The four conditions are mutual exclusion, hold-and-wait, no pre-emption and circular wait. The semaphore solution breaks circular wait with the room semaphore (at most 3 contenders). The monitor solution breaks hold-and-wait by granting both forks atomically or none. The Readers-Writers programs hold only one resource lock, so they cannot deadlock.

Starvation.

- **Readers-Writers (semaphore):** the turnstile queues requests, so neither readers nor writers should starve. Python semaphores are not guaranteed to wake threads in strict first-in-first-out order, so this is not formally proved.
- **Readers-Writers (monitor):** writer preference means readers could starve if writers kept arriving without a break.
- **Philosophers (semaphore):** the room semaphore is also not strictly first-in-first-out, so starvation is unlikely but not excluded.
- **Philosophers (monitor):** a philosopher could starve if its two neighbours keep taking turns eating so that it never sees both free. A fix is to record how long each philosopher has waited and give priority to the oldest.

10. Comparison of Semaphore and Monitor Approaches

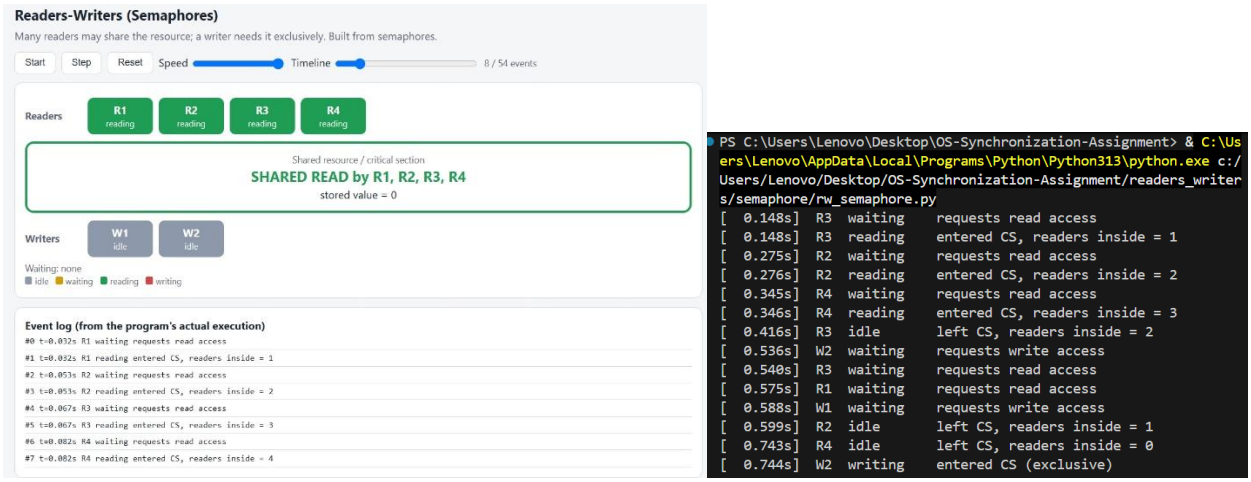
Point	Semaphores	Monitors
Level	Low-level counters; programmer must pair every wait with a signal	Higher-level; lock and conditions are packaged in one class
Error risk	A forgotten signal or wrong order can deadlock or break exclusion	Mutual exclusion is automatic inside the entry procedures; mistakes are mainly wrong conditions
Waiting on a condition	Indirect, using counters and extra semaphores (turnstile, room)	Direct, using condition variables with wait_for
Readability	State is spread over several semaphores	State and rules sit together, easier to explain
Flexibility	Very flexible, but harder to reason about	Easy to change policy (for example writer priority)
Language support	Available almost everywhere	Needs lock plus conditions; built into some languages

In this assignment the monitor solutions were easier to read and check, while the semaphore solutions showed more clearly how the low-level mechanism works.

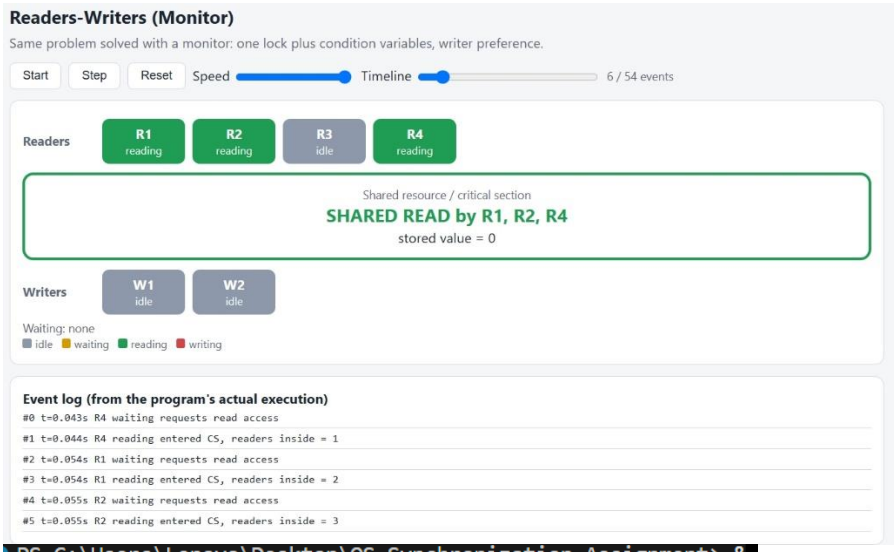
11. Screenshots

Insert your own screenshots here, taken from your runs:

- **Figure 1:** terminal output of rw_semaphore.py and the generated HTML simulation



- **Figure 2:** terminal output of rw_monitor.py and the generated HTML simulation



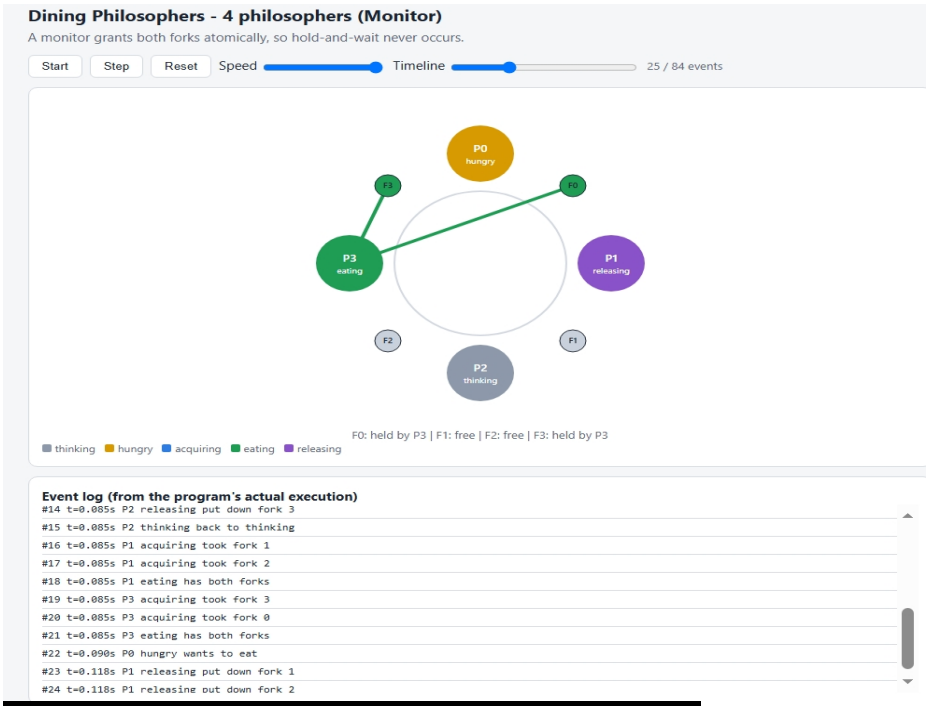

```
PS C:\Users\Lenovo\Desktop\OS-Synchronization-Assignment> &
python.exe c:/Users/Lenovo/Desktop/OS-Synchronization-Assign
[  0.111s]  R1  waiting    requests read access
[  0.112s]  R1  reading    entered CS, readers inside = 1
[  0.145s]  R2  waiting    requests read access
[  0.146s]  R2  reading    entered CS, readers inside = 2
[  0.423s]  R3  waiting    requests read access
[  0.423s]  R3  reading    entered CS, readers inside = 3
[  0.484s]  R4  waiting    requests read access
[  0.484s]  R4  reading    entered CS, readers inside = 4
[  0.509s]  R2  idle       left CS, readers inside = 3
[  0.598s]  R1  idle       left CS, readers inside = 2
[  0.782s]  W2  waiting    requests write access
[  0.819s]  R3  idle       left CS, readers inside = 1
[  0.839s]  R4  idle       left CS, readers inside = 0
[  0.839s]  W2  writing    entered CS (exclusive)
```

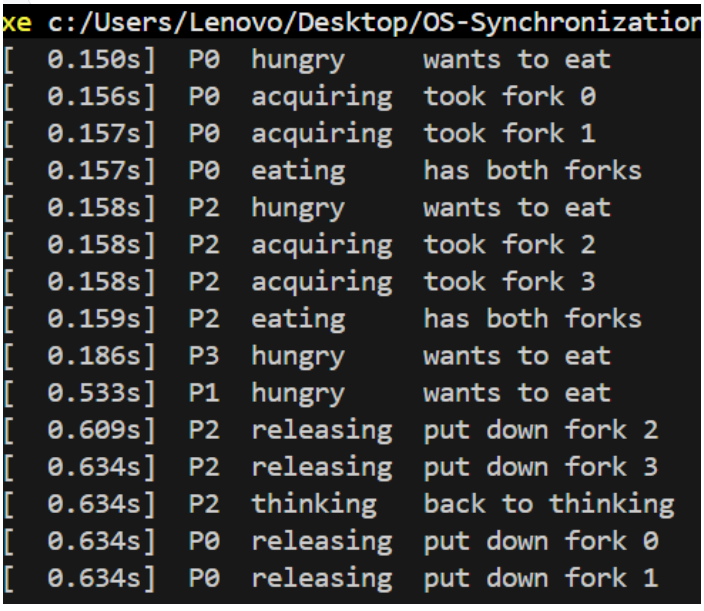
- **Figure 3:** terminal output of dp_semaphore.py and the generated HTML simulation




```
xe c:/Users/Lenovo/Desktop/OS-Synchronization-Assignment/dining_p
[  4.299s]  P2  releasing  put down fork 2
[  4.300s]  P2  releasing  put down fork 3
[  4.301s]  P2  thinking   finished all meals
[  4.301s]  P1  acquiring  picked up fork 2
[  4.302s]  P1  eating     has both forks
[  4.632s]  P1  releasing  put down fork 1
[  4.633s]  P1  releasing  put down fork 2
[  4.634s]  P1  thinking   finished all meals
[  4.635s]  P0  acquiring  picked up fork 1
[  4.635s]  P0  eating     has both forks
[  4.943s]  P0  releasing  put down fork 0
[  4.944s]  P0  releasing  put down fork 1
[  4.944s]  P0  thinking   finished all meals
validation OK: forks never shared and every meal had both forks
```

- **Figure 4:** terminal output of dp_monitor.py and the generated HTML simulation





12. AI Usage Summary

The AI tool used was Claude (Anthropic) in the claude.ai chat. I gave it the assignment PDF and asked for code for the four problems. It produced the four programs and the shared HTML generator. I then asked follow-up questions about opening and running the files, fixed a setup error (running a file from inside the zip, so the common folder was missing), and asked for the HTML to open automatically after a run, which was added. The AI also declined my request to help avoid AI detection and advised that AI use must be disclosed. I ran the programs and checked the output and the HTML. A limitation I noticed is that the first version did not open the HTML automatically. The full prompts are in ai_prompts/prompts.md.

13. References

1. Silberschatz, A., Galvin, P. B. and Gagne, G., Operating System Concepts, 10th ed., Wiley, 2018.
2. Tanenbaum, A. S. and Bos, H., Modern Operating Systems, 4th ed., Pearson, 2014.
3. Downey, A. B., The Little Book of Semaphores, Green Tea Press.
4. Courtois, P. J., Heymans, F. and Parnas, D. L., 'Concurrent Control with Readers and Writers', Communications of the ACM, 14(10), 1971.
5. Hoare, C. A. R., 'Monitors: An Operating System Structuring Concept', Communications of the ACM, 17(10), 1974.
6. Dijkstra, E. W., 'Hierarchical Ordering of Sequential Processes', Acta Informatica, 1, 1971.
7. Python Software Foundation, documentation of the threading and webbrowser modules, docs.python.org/3.
8. Anthropic, Claude (AI assistant), claude.ai, used for code generation and explanations, October 2026.

14. Academic Integrity Declaration

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

Signature: 

Date: 13/10/2026